Subword Tokenizer and SLM Training/Evaluation Stack: A Practical Foundation for Physics Research Assistant Systems

Jaydip Singh Linkan Kumbhar

July 13, 2026

Abstract

This paper presents an implementation-focused foundation for building a domain-oriented small language model (SLM) system for physics research assistance. The system combines a production Rust Byte Pair Encoding (BPE) tokenizer with a Python-based training and evaluation pipeline designed for resource-constrained hardware (Apple Silicon/MPS). We report Phase 1–3 results: baseline TinyLM loss 0.0107, best LoRA loss 0.0060 (44% improvement), held-out test loss 0.00595 (45.45% gain vs baseline), and improved physics QA score (+0.125 absolute). We document architecture, operational scripts, reliability mechanisms (checkpointing and stream retry/backoff), and a phased roadmap toward retrieval-augmented generation (RAG) and tool-using agents.

1 Introduction

Small language models are increasingly relevant for cost-efficient, low-latency, and on-device AI workloads. Our objective is to establish a robust foundation layer that is reproducible on laptop-class hardware while remaining directly extensible to larger-scale training and deployment.

The current system has two aligned layers:

1. **Tokenizer layer (Rust):** a production BPE tokenizer with CLI and reusable library APIs.
2. **Prototype training layer (Python):** a TinyLM training path using the Rust tokenizer model, with automatic checkpointing and evaluation.

2 System Architecture

The implementation follows a streaming-to-training architecture:

1. Live arXiv stream with retry/backoff for transient failures.
2. Tokenization through Rust CLI (`bpe-tokenizer`) using a 32K vocabulary model.
3. Tiny Transformer training (MPS/CUDA/CPU auto-selection).
4. Step checkpoints, final checkpoints, and multi-run ranking by evaluation loss.

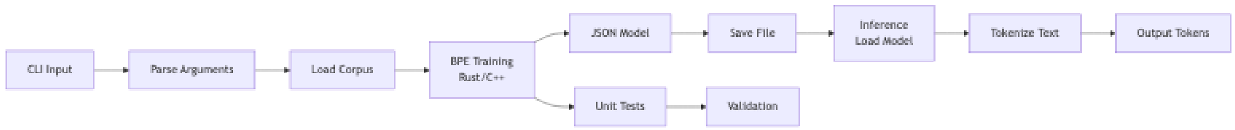

Figure 1: System architecture: Live arXiv stream → Rust tokenizer → TinyLM training → checkpoint evaluation.

3 Implementation Details

3.1 Tokenizer Commands

Current tokenizer workflow:

- `cargo run -bin bpe-tokenizer - train <corpus.txt> <vocab_size>`
- `cargo run -bin bpe-tokenizer - expand <corpus.txt> <vocab_size>`
- `cargo run -bin bpe-tokenizer - tokenize <text>`
- `cargo run -bin bpe-tokenizer - decode <comma_separated_ids>`
- `cargo run -bin bpe-tokenizer - count`

3.2 Prototype Training Commands

- Quick prototype: `./scripts/run_prototype.sh`
- 3-run compare + best select: `./scripts/run_prototype_3x_and_select_best.sh`
- Long run (∼3–4h on M3 Pro): `./scripts/run_prototype_long_4h.sh`
- Evaluation: `./scripts/run_evaluation.sh`

3.3 Validation and Reliability

- Rust tokenizer tests are passing.
- Prototype training is validated on Mac MPS.
- Best-checkpoint JSON reports are generated automatically.
- Retry/backoff mitigates arXiv timeout interruptions in long runs.

4 Roadmap to Publication-Grade Physics Assistant SLM

4.1 Final Goal

Build a physics research assistant using a 3B–7B parameter SLM, LoRA fine-tuning, RAG grounding, and tool-using agents.

4.2 Phase Plan

Weeks	Phase	Deliverable
1–2	Data Pipeline	300M–1B tokenized physics corpus
3–5	Base SLM Training	3B–7B checkpoint (decoder-only baseline)
6–7	LoRA Fine-Tuning	Parameter-efficient domain adaptation weights
8–9	RAG Integration	Vector index + retrieval-augmented response path
10–11	Agent Layer	Tool-using multi-step reasoning workflow
12+	Deployment	API + optional UI + cloud/runtime packaging

4.3 Success Criteria

- Physics-grounded coherent answers with retrieved context.
- Stable tool-calling behavior for equations and code tasks.
- End-to-end response latency below 5 seconds in target deployment profile.

5 Risks and Mitigations

- **Streaming instability:** addressed via retry/backoff and periodic checkpoints.
- **Artifact growth:** requires retention and storage policy.
- **Documentation drift:** enforce CI-based PDF generation from source.

6 Conclusion

This work establishes a practical and reproducible foundation for SLM development, balancing implementation realism with a clear path to research-grade publication outcomes. The current milestone is a validated tokenizer+training+evaluation stack (Phase 1–3 complete); the next milestones are generation-control tuning, RAG integration, and agent-layer deployment.

References

[1] P. Jhandi, O. Kazi, S. Subramanian, and N. Sendas, "Small Language Models for Efficient Agentic Tool Calling: Outperforming Large Models with Targeted Fine-tuning," arXiv:2512.15943, 2025.

[2] Y. Kang et al., "Fine-tuning Small Language Models as Efficient Enterprise Search Relevance Labelers," arXiv:2601.03211, 2026.

[3] R. Sharma and M. Mehta, "Small Language Models for Agentic Systems: A Survey of Architectures, Capabilities, and Deployment Trade-offs," arXiv:2510.03847, 2025.

[4] Z. K. Chong et al., "Compiling Deterministic Structure into SLM Harnesses," arXiv:2604.17450, 2026.

[5] A. Karpathy, "NanoGPT: The simplest, fastest repository for training medium-sized GPTs," GitHub, 2023.

[6] R. Eldan and Y. Li, "TinyStories: How Small Can Language Models Be and Still Speak Coherently?," arXiv:2305.07759, 2023.